

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1001

COURSE NAME: Software Engineering

Name : Dhuneshwaran V

Reg.no : 192324169

Problem 3: Choosing Architecture for a Scalable Food Delivery Platform

Scenario:

A startup wants to build a food delivery platform like Swiggy/Zomato that can scale during peak hours and process thousands of orders per minute. The CTO is unsure whether to use microservices, serverless, or a monolithic architecture.

Task:

Recommend an architecture, justify the decision, explain the role of event-driven architecture, and list suitable microservices.

Deliverables:

- Recommended architecture choice
- Three technical reasons for the choice
- Explanation of event-driven architecture benefits
- List of three microservices (e.g., Orders, Delivery, Payments)

Reasoning:

This activity simulates real-world architectural decision-making in highly scalable products

1. Executive Summary

Food delivery platforms operate under extreme and highly variable load. Order volume can spike 5-10x during lunch and dinner windows, weekends, and promotional events, while remaining low for the rest of the day. The system must coordinate several independent workflows in real time — order placement, restaurant acceptance, payment capture, delivery-partner assignment, and live tracking — each with different scaling, latency, and reliability characteristics.

This report recommends a microservices architecture, built around an event-driven backbone, as the primary system design, with serverless functions used selectively for bursty, stateless workloads (e.g., notifications, image processing, invoice generation). A pure monolithic architecture is not recommended beyond the earliest MVP stage, and a pure serverless architecture is not recommended as the sole approach for core order-processing workflows.

2. Recommended Architecture

Recommendation: Microservices architecture with an event-driven backbone (event streaming platform such as Apache Kafka or AWS Kinesis), complemented by serverless functions for auxiliary, spiky workloads.

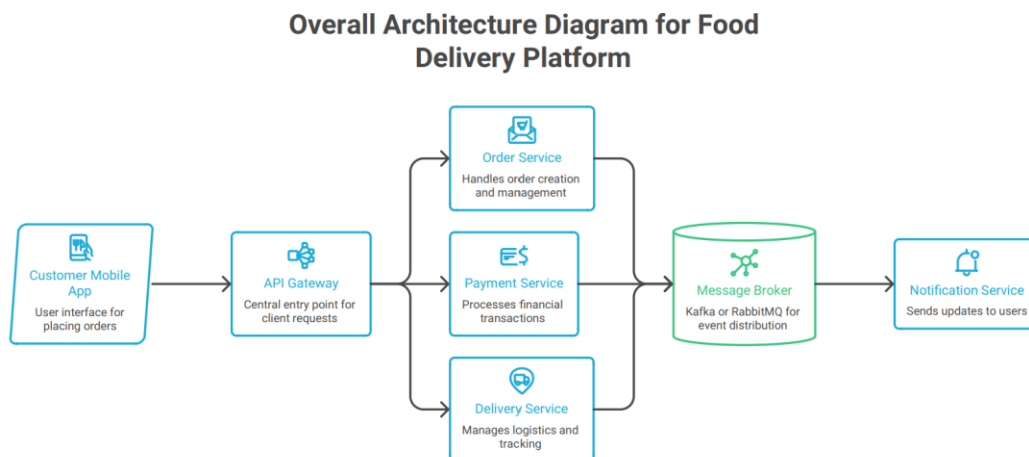
Each core business capability — orders, payments, delivery, restaurants, notifications, and customer accounts — is implemented as an independently deployable service, owning its own data store. Services communicate primarily through asynchronous events rather than direct synchronous calls, which decouples them and allows each to scale independently based on its own load profile.

2.1 Why Not a Monolith

A monolithic architecture bundles all functionality into a single deployable unit and a single (typically relational) database. This is simple to build initially, but at food-delivery scale it becomes a bottleneck: a spike in order volume forces the entire application to scale even if only the ordering module is under load, a bug in the ratings module can bring down checkout, and every deployment risks the whole platform. Monoliths are appropriate for validating an idea quickly, but not for a platform expected to process thousands of orders per minute with independent, fast-moving teams.

2.2 Why Not Pure Serverless

A fully serverless approach (e.g., functions-as-a-service for every workflow) offers excellent automatic scaling and low idle cost, but introduces challenges for this domain: cold-start latency can hurt real-time experiences like live order tracking, long-running or stateful workflows (an active delivery that persists for 20-40 minutes) are awkward to model as short-lived functions, and per-invocation pricing can become expensive at sustained high throughput compared to always-on services. Serverless is therefore used tactically — for notification fan-out, receipt/invoice generation, image resizing, and scheduled jobs — rather than as the backbone for order and payment processing.



3. Three Technical Reasons for the Choice

3.1 Independent, Fine-Grained Scalability

Order intake, delivery-partner matching, and payment processing each experience different load patterns. Microservices allow each capability to be scaled independently — for example, running many more instances of the Order Service and Delivery-Matching Service during a lunch-hour surge, while keeping the Ratings or Reporting services at baseline capacity. This avoids over-provisioning the entire system just to handle load on one hot path, directly reducing infrastructure cost while meeting peak throughput of thousands of orders per minute.

3.2 Fault Isolation and Resilience

In a monolith, a failure or slow dependency in one module (for example, a third-party payment gateway timing out) can exhaust threads or memory and take down the entire application. With microservices, a failure in the Payment Service can be contained using timeouts, circuit breakers, and retries, while Order placement, Search, and Restaurant browsing continue to function. This isolation is critical for a platform where downtime during a peak dinner rush directly translates to lost revenue and customer trust.

3.3 Independent Deployability and Team Autonomy

Microservices allow separate teams to own, build, test, and deploy the Orders, Payments, Delivery, and Restaurant services independently, using the technology stack best suited to each (e.g., a geospatial-optimized stack for delivery routing, a strongly consistent relational store for payments). This shortens release cycles, reduces the blast radius of any single deployment, and lets the platform evolve quickly — for example, rolling out a new delivery-assignment algorithm without redeploying or risking the payment flow.

4. Role and Benefits of Event-Driven Architecture

An event-driven architecture (EDA) uses an event broker (e.g., Kafka, AWS Kinesis, or RabbitMQ) as the central nervous system connecting microservices. Instead of the Order Service directly and synchronously calling the Payment, Delivery, and Notification services, it publishes an event such as `OrderPlaced`. Every interested service subscribes to the events it cares about and reacts independently.

4.1 Key Benefits

- **Loose coupling:** Services do not need to know about each other's existence or APIs directly — they only need to agree on event schemas. This makes it easy to add a new consumer (e.g., a fraud-detection service) without changing the Order Service.
- **Real-time responsiveness:** Events allow live updates to propagate immediately to consumers such as the customer app (order status), delivery-partner app (new assignment), and restaurant dashboard (incoming order) — essential for the real-time feel users expect from food delivery apps.
- **Elastic handling of traffic spikes:** The event broker acts as a durable buffer. During a peak surge, incoming order events queue safely in the stream and are processed by consumer services at a sustainable rate, preventing sudden overload or dropped requests.
- **Asynchronous, resilient workflows:** If the Notification Service is temporarily down, `OrderPlaced` events remain in the stream and are processed once it recovers — no orders are lost, unlike a synchronous call that would simply fail.
- **Auditability and replay:** Every state change (order placed, accepted, picked up, delivered) is captured as an immutable event, giving a natural audit trail and the ability to replay events to rebuild state, debug incidents, or feed analytics and machine-learning pipelines.
- **Simplified orchestration of multi-step workflows:** A single order touches many services (inventory check, payment, delivery assignment, notifications). Event choreography lets each service react to relevant events independently, avoiding a fragile central orchestrator that must know every downstream step.

5. Suitable Microservices

The table below lists the core microservices for the platform, their primary responsibility, and the key events each one publishes or consumes.

Microservice	Responsibility	Key Events Published / Consumed
Orders Service	Handles cart, order creation, order status, and lifecycle management from placement to completion.	Publishes: OrderPlaced, OrderCancelled. Consumes: PaymentConfirmed, DeliveryAssigned.
Payments Service	Processes payment authorization, capture, refunds, and integrates with external payment gateways.	Consumes: OrderPlaced. Publishes: PaymentConfirmed, PaymentFailed.
Delivery / Fleet Service	Matches orders to available delivery partners, tracks live location, and manages delivery lifecycle.	Consumes: PaymentConfirmed. Publishes: DeliveryAssigned, DeliveryPickedUp, DeliveryCompleted.
Restaurant Service	Manages restaurant menus, availability, pricing, and order acceptance/rejection by the kitchen.	Consumes: OrderPlaced. Publishes: OrderAccepted, OrderRejected.
Notification Service	Sends push notifications, SMS, and in-app alerts to customers, restaurants, and delivery partners.	Consumes: OrderPlaced, PaymentConfirmed, DeliveryAssigned, DeliveryCompleted.
User / Account Service	Manages customer, restaurant, and delivery-partner profiles, authentication, and preferences.	Publishes: UserRegistered, ProfileUpdated. Consumed by most services for identity checks.

Beyond these six, a production platform typically adds supporting services such as Search & Discovery, Ratings & Reviews, Pricing & Promotions, and Analytics/Reporting — each following the same principle of a single, well-bounded responsibility communicating through events.

6. Conclusion

A microservices architecture, connected through an event-driven backbone and supplemented by serverless functions for bursty auxiliary tasks, gives the platform the independent scalability, fault isolation, and real-time responsiveness required to reliably process thousands of orders per minute. This approach trades some upfront operational complexity (service orchestration, distributed monitoring, event-schema governance) for long-term agility and resilience — a trade-off that is well justified once the platform moves beyond an early MVP and needs to handle production-scale, peak-hour demand.